Framework-Topology Visualization Adapter Specification

Projected framework graphs, retained atomic-path diagnostics, and 2-D/3-D rendering wrappers

mdstats

Contents

Purpose and status

This document is the normative specification for `mdstats.plotting.framework_topology_graph`, introduced in `mdstats` 0.13.0, extended with compact node-display modes in 0.13.1, and advanced to orientation-aware path metadata in 0.15.0. The module adapts an authoritative `FrameworkTopology` into the generic visualization pipeline without reconstructing atomic connectivity or repeating framework path projection.

The adapter supports two complementary views:

1. a **projected framework graph**, in which retained framework atoms are nodes and contracted linker paths are decorated multigraph edges;
2. an **atomic-path diagnostic graph**, in which each projected edge is expanded into the exact atomic path already stored by `FrameworkTopology`.

Both views are renderer-independent `DecoratedGraphView` objects and may be sent to the existing Matplotlib 2-D and Plotly 3-D renderers.

Motive

Ring enumeration will consume the projected framework graph rather than the raw atomic graph. Numerical counts alone cannot expose every projection error. A graph with the expected number of vertices and edges may still contain a wrong linker, incorrect periodic translation, duplicated path, or misplaced parallel edge.

The visualization adapter therefore provides a direct diagnostic of the exact graph that later ring modules will receive. It must preserve scientific identity and periodic winding while remaining strictly downstream of framework projection.

Normative ownership

This specification owns:

- conversion from `FrameworkTopology` to `DecoratedGraphView`;
- projected and atomic-path display modes;
- frame-local periodic image-shift reconstruction;
- framework-specific graph metadata;
- `FrameworkPathSegmentKey` identity;
- framework-specific default style presets;
- 2-D and 3-D convenience wrappers;
- the relaxed Na-LTA visualization acceptance fixture.

It does not own:

- atomic connectivity;
- role resolution or framework path search;
- projected graph canonicalization;
- periodic display materialization;
- generic filtering, layout, styling rules, or rendering;
- ring enumeration.

AI context summary

- `FrameworkTopology` is authoritative. The adapter never decides which framework edge exists.
- Projected node keys are original framework-vertex atom indices.
- Projected edge keys are authoritative `FrameworkEdgeKey` objects.
- Atomic-path mode uses only `FrameworkEdgePath.atomic_path_indices` and stored periodic path metadata; no path search is repeated.
- Canonical projected shifts cannot be drawn directly against arbitrary wrapped frame coordinates. The adapter reconstructs frame-consistent shifts and verifies winding equivalence.
- Diagnostic path segments use stable `FrameworkPathSegmentKey` objects and retain their parent projected-edge identity.
- Spectators and excluded atoms are not displayed unless a future adapter explicitly adds them.
- Renderers remain generic and contain no framework chemistry.

Public module

`mdstats/plotting/framework_topology_graph.py`

The symbols are exported from both `mdstats.plotting` and the package root.

Display mode

```
from enum import Enum

class FrameworkGraphDisplayMode(str, Enum):
    PROJECTED = "projected"
    ATOMIC_PATHS = "atomic_paths"
```

`PROJECTED` shows the contracted framework multigraph.

`ATOMIC_PATHS` expands each projected edge into its retained atomic path. It is a visual diagnostic of the projection result, not a replacement for the authoritative atomic-connectivity graph.

String values may be accepted where the enum is expected. Unknown values are errors.

Diagnostic segment identity

```
from dataclasses import dataclass

@dataclass(frozen=True, order=True, slots=True)
class FrameworkPathSegmentKey:
    framework_edge_key: FrameworkEdgeKey
    segment_index: int
    atom_i: int
    atom_j: int

    def to_dict(self) -> dict[str, object]: ...
    @classmethod
    def from_dict(cls, payload: Mapping[str, object]) -> "FrameworkPathSegmentKey": ...
```

Constraints:

- `framework_edge_key` is an authoritative projected-edge key;
- `segment_index` is a nonnegative position in the parent atomic path;
- `atom_i` and `atom_j` are nonnegative canonical atom indices;
- `atom_i != atom_j`;
- the key is immutable, hashable, sortable, and round-trip serializable.

A diagnostic edge is unique by parent projected edge and segment position. Two projected edges may therefore retain separate diagnostic segments even if those segments connect the same canonical atom pair.

Public adapter

```
def graph_view_from_framework_topology(
    collection: AtomisticFrameCollection,
    topology: FrameworkTopology,
    *,
    frame_index: int,
    display_mode: FrameworkGraphDisplayMode | str =
        FrameworkGraphDisplayMode.PROJECTED,
) -> DecoratedGraphView:
    ...
```

Input types

`collection` An `AtomisticFrameCollection` with fixed atom identity ordering.
`topology` One immutable `FrameworkTopology` produced by `build_framework_topology`.
`frame_index` A collection-frame position used only for Cartesian coordinates and frame-local display geometry.
`display_mode` Either `PROJECTED` or `ATOMIC_PATHS`.

Required input constraints

The adapter must verify:

- `collection` has a valid selected frame;
- `topology` is a `FrameworkTopology`;
- every vertex and retained atomic-path atom index exists in the collection;
- atomic numbers in `topology.resolved_roles` match the collection ordering;
- topology and collection PBC flags agree;
- the selected cell is finite and nonsingular;
- selected wrapped positions are finite;
- all canonical and reconstructed shifts are zero on nonperiodic axes;
- the selected frame is topologically compatible with the stored projected graph.

A mismatch raises `GraphAdapterError`. The adapter must not remap atom identities, change roles, delete edges, or infer a replacement topology.

Projected framework view

Node identity and geometry

Projected node keys are:

```
tuple(int(atom) for atom in topology.vertex_atom_indices)
```

Node positions are the selected frame's wrapped Cartesian positions of those atoms.

Projected edge identity

Projected edge keys are:

```
topology.edge_keys
```

The graph is undirected and may be a multigraph. Parallel edges and periodic self-image edges remain distinct. Undirected adjacency does not erase path orientation: every edge retains one canonical ordered atomic path and one exact reverse traversal.

Projected node attributes

The view must provide at least:

```
atom_index                int
atomic_number             int
symbol                    str
framework_role            "vertex"
framework_vertex_index    int
```

```
projected_degree           int
component_id               int
```

`framework_vertex_index` is the dense position in `topology.vertex_atom_indices`; it is not scientific identity.

Projected edge attributes

The view must provide at least:

```
vertex_i                        int
vertex_j                        int
source_symbol                   str
target_symbol                   str
species_pair                    tuple[str, str]
rule_id                         str
edge_kind                       str
atomic_path_indices             tuple[int, ...]
reverse_atomic_path_indices     tuple[int, ...]
canonical_path_symbols          tuple[str, ...]
reverse_path_symbols            tuple[str, ...]
canonical_orientation           "vertex_i_to_vertex_j"
orientation_aware               bool
internal_linker_indices         tuple[int, ...]
internal_linker_symbols         tuple[str, ...]
linker_count                    int
path_segment_count              int
raw_image_shift                 tuple[int, int, int]
canonical_image_shift           tuple[int, int, int]
display_image_shift             tuple[int, int, int]
periodic                         bool
parallel_multiplicity            int
parallel_rank                    int
```

`parallel_multiplicity` counts projected edges sharing the same unordered endpoint pair. `parallel_rank` is the deterministic zero-based position after canonical edge sorting.

For an asymmetric bridge, `canonical_path_symbols` and `reverse_path_symbols` show the two traversal descriptions of one undirected edge. For example, `("Si", "O", "S", "Al")` reverses to `("Al", "S", "O", "Si")`; it must not be displayed as `("Si", "S", "O", "Al")`.

Atomic-path diagnostic view

Purpose

Atomic-path mode visualizes exactly how each projected edge was obtained. For a single oxygen bridge,

$$\mathrm{T}_i - \mathrm{O}_k - \mathrm{T}_j,$$

the diagnostic graph contains the two stored path segments rather than one direct projected line.

Node set

The diagnostic node set is the sorted union of:

- all framework vertices;
- all linker atoms appearing in accepted projected edges.

Spectator, excluded, unused linker, and unrelated active atoms are omitted.

Node keys remain canonical atom indices.

Diagnostic node attributes

The view must provide at least:

```
atom_index                          int
atomic_number                       int
symbol                              str
framework_role                      "vertex" | "linker"
framework_vertex_index              int, -1 for linkers
projected_degree                    int, 0 for linkers
linker_framework_degree             int, 0 for vertices
projected_path_membership_count      int
component_id                        int
```

A used linker inherits the component of its parent projected edge. A linker assigned to multiple incompatible projected components is an adapter error.

Diagnostic edges

Each consecutive pair in every retained atomic path becomes one multigraph edge with a `FrameworkPathSegmentKey`.

Required attributes:

```
segment_index                   int
atom_i                          int
atom_j                          int
source_symbol                   str
target_symbol                   str
species_pair                    tuple[str, str]
segment_kind                    str
parent_vertex_i                 int
parent_vertex_j                 int
parent_rule_id                  str
parent_edge_kind                str
parent_atomic_path_indices      tuple[int, ...]
parent_canonical_image_shift    tuple[int, int, int]
canonical_segment_image_shift   tuple[int, int, int]
display_image_shift             tuple[int, int, int]
periodic                        bool
```

`segment_kind` is one of:

```
direct_vertex
vertex_linker
linker_linker
linker_vertex
```

The diagnostic graph is a multigraph because separate projected edges may retain geometrically overlapping atomic segments. Segment order follows the canonical stored path. Reverse traversal metadata is retained at the parent-edge level rather than represented as duplicate diagnostic edges.

Periodic display geometry

Canonical projected shifts

For a projected edge stored from canonical endpoint i to j, let $\mathbf{M}_{ij}^{c}$ be `FrameworkEdgeKey.image_shift`. The reverse traversal uses $-\mathbf{M}_{ij}^{c}$ and reverses the complete atomic path. The stored orientation is a canonical representation, not a physical directed arrow.

The selected frame uses wrapped coordinates that may differ by an integer gauge. Let $\mathbf{M}_{ij}^{f}$ be the frame-consistent shift and $\mathbf{g}_i$ the vertex gauge:

$$\mathbf{M}_{ij}^{c} = \mathbf{M}_{ij}^{f} + \mathbf{g}_i - \mathbf{g}_j.$$

Therefore,

$$\mathbf{M}_{ij}^{f} = \mathbf{M}_{ij}^{c} - \mathbf{g}_i + \mathbf{g}_j.$$

Frame-local atomic path shift

The selected-frame shift of a stored path

$$v_0 \to v_1 \to \cdots \to v_k$$

is obtained by computing a minimum-image shift for each retained atomic segment and summing:

$$\mathbf{M}^{\mathrm{path}}_{v_0 v_k} = \sum_{\ell=0}^{k-1} \boldsymbol{\mu}_{v_\ell v_{\ell+1}}.$$

This operation reconstructs display geometry only. The atom sequence is already stored and must not be searched or altered. When a consumer requests reverse traversal, it must use `edge.oriented(-1)` so atom order, linker order, and periodic translations reverse together.

Projected-shift reconstruction algorithm

The adapter must:

1. compute frame-local path shifts for every projected edge;
2. build a deterministic spanning forest of projected vertices;
3. root each component at its smallest atom index;
4. propagate the gauge using tree-edge path shifts;
5. reconstruct every projected edge from canonical graph shifts;
6. verify reconstructed shifts equal the selected-frame path sums;
7. preserve non-tree periodic winding;
8. warn on minimum-image ties;
9. reject nonperiodic-axis violations or winding inconsistencies.

The equality check is important. It prevents a topology from being drawn against an unrelated or topologically incompatible frame.

Diagnostic segment shifts

Atomic-path diagnostic edges use the frame-local minimum-image shift of each stored segment. Their canonical segment shift is derived from the authoritative lifted path offsets:

$$\mathbf{m}^c_\ell = \mathbf{q}_{\ell+1} - \mathbf{q}_\ell,$$

where

$$(\mathbf{q}_0, \ldots, \mathbf{q}_k) = (\mathbf{0}, \text{internal linker image offsets}, \mathbf{M}^c_{v_0 v_k}).$$

The sum of canonical segment shifts must equal the parent canonical projected shift. The sum of display segment shifts must equal the parent frame-consistent projected shift.

Adapter metadata

Both views must include:

```
adapter_schema_version
collection_frame_index
frame_id
display_mode
source_framework_graph_digest
source_framework_topology_digest
source_connectivity_digest
mapping_digest
n_source_vertices
n_source_edges
frame_shift_reconstruction
adapter_warnings
```

Projected views should additionally report parallel and self-image edge counts. Diagnostic views should report displayed linker and segment counts.

Default framework style

`GraphStyle` gains:

```
@classmethod
def framework_default(
    cls,
    palette: ChemicalColorPalette | None = None,
    *,
    diagnostic: bool = False,
    node_display_mode: NodeDisplayMode | str = NodeDisplayMode.MARKERS,
    node_dot_size: float = 16.0,
) -> "GraphStyle":
    ...
```

Projected style

Default projected styling should provide:

```
Si vertices          blue
Al vertices          cyan
generic vertices     neutral gray
direct edges         dark gray
oxygen bridges       red
sulfur bridges       yellow-orange
unknown edge kinds   medium gray
```

Projected edges should be thick enough to remain visible in 2-D and 3-D without obscuring node positions.

Diagnostic style

With `diagnostic=True`:

- vertex colors remain species based;
- linker nodes are smaller than framework vertices;
- segment edges use endpoint color transitions;
- path segments are thinner than projected framework edges;
- legends distinguish vertex and linker roles when practical;
- `node_display_mode="dots"` preserves species colors using small points;
- `node_display_mode="hidden"` suppresses all node markers for edge-only views.

Styling affects no scientific identity.

Compact framework-node modes

Large periodic framework views may be rendered with reduced vertex clutter while retaining the same projected topology and species colors:

```
GraphStyle.framework_default(node_display_mode="dots")
GraphStyle.framework_default(node_display_mode="hidden")
```

`dots` keeps Si, Al, and other vertex colors but replaces full markers by small circular points. `hidden` produces an edge-only view: no vertex or ghost markers, no node labels, and no node legend entries are created. In both cases the returned `GraphRenderResult` or `InteractiveGraphRenderResult` still reports all scientific node keys and the unchanged edge endpoints.

For atomic-path diagnostics, hidden nodes may make the path chemically ambiguous when edge colors are constant. The default diagnostic style uses endpoint-split colors, so hidden path nodes still influence segment colors; nevertheless, full markers or dots are preferred when linker identity must be inspected explicitly.

Two-dimensional wrapper

```
def plot_framework_topology_2d(
    collection: AtomisticFrameCollection,
    topology: FrameworkTopology,
    *,
    frame_index: int,
    display_mode: FrameworkGraphDisplayMode | str =
        FrameworkGraphDisplayMode.PROJECTED,
    layout: GraphLayoutOptions | None = None,
    style: GraphStyle | None = None,
    focus: GraphFocus | None = None,
    graph_filter: GraphFilter | None = None,
    complexity_policy: GraphComplexityPolicy | None = None,
    periodic: PeriodicDisplayOptions | None = None,
    options: Graph2DRenderOptions | None = None,
    axes: matplotlib.axes.Axes | None = None,
) -> GraphRenderResult:
    ...
```

The wrapper adapts once and delegates to `plot_decorated_graph_2d`.

Default style:

```
GraphStyle.framework_default(
    diagnostic=(display_mode is FrameworkGraphDisplayMode.ATOMIC_PATHS)
)
```

Interactive three-dimensional wrapper

```
def plot_framework_topology_3d(
    collection: AtomisticFrameCollection,
    topology: FrameworkTopology,
    *,
    frame_index: int,
    display_mode: FrameworkGraphDisplayMode | str =
        FrameworkGraphDisplayMode.PROJECTED,
    periodic: PeriodicDisplayOptions | None = None,
    style: GraphStyle | None = None,
    focus: GraphFocus | None = None,
    graph_filter: GraphFilter | None = None,
    complexity_policy: GraphComplexityPolicy | None = None,
    options: Graph3DRenderOptions | None = None,
) -> InteractiveGraphRenderResult:
    ...
```

The wrapper delegates to `plot_decorated_graph_3d`. Plotly remains optional.

Interactive HTML output is an artifact. It should not be embedded automatically in text interfaces where large inline scenes may overload the client.

Complexity and filtering

The generic `GraphComplexityPolicy` applies after adaptation and before periodic materialization.

The adapter must not silently:

- sample framework vertices;
- omit projected edges;
- merge parallel paths;
- collapse diagnostic segments;
- hide periodic winding;
- replace atomic paths by straight-line guesses;

- treat hidden nodes as removed scientific objects.

Users may explicitly focus or filter the resulting view through the generic graph API.

Edge cases and warnings

Parallel projected paths

Parallel edges are valid scientific objects. The view sets `multigraph=True`. The current renderers may require explicit overlap permission when two paths project to exactly the same geometric line.

Asymmetric linker paths

The projected graph remains undirected, but an asymmetric path retains ordered canonical and reverse signatures. Hover data and serialized graph attributes must make both available. The adapter must never independently sort linker symbols or combine `A-O-S-B` with `A-S-O-B`.

Self-image projected edges

A nonzero self-image edge is valid in a periodic quotient graph. Generic renderers currently reject or incompletely represent self-loops. The adapter preserves the edge and allows the renderer to issue the appropriate unsupported-feature error.

Shared linker atoms

A linker may appear in multiple projected edges in unusual or defective structures. Atomic-path mode uses one canonical linker node and multiple stable segment edges. The adapter must not duplicate scientific linker identity merely to avoid overlap.

Branching or dangling linkers

Only linkers already used by accepted projected paths appear in atomic-path mode. Unused, dangling, or branching linker diagnostics remain available from `FrameworkProjectionReport`; they are not silently inserted as framework edges.

Minimum-image ties

A bond segment exactly at a half-cell boundary has an ambiguous display image. The adapter should retain deterministic behavior and record a warning. Scientific graph identity remains unchanged.

Frame mismatch

If frame-local path shifts cannot be reconciled with canonical projected winding, the adapter raises `GraphAdapterError`. Drawing a plausible but incorrect graph is not acceptable.

Strained cells

The cell matrix and selected-frame Cartesian coordinates are used directly. The renderer must preserve the actual triclinic metric; the adapter must not orthogonalize or otherwise reshape the cell.

Na-LTA acceptance fixture

The relaxed Na-LTA structure contains:

```
Si24 Al24 O96 Na24
168 atoms total
```

The authoritative framework topology contains:

```
48 framework vertices
96 projected T-T edges
24 Si vertices
24 Al vertices
degree 4 at every framework vertex
one projected component
```

Projected acceptance view

The projected view must contain:

```
48 display-source nodes
96 display-source edges
multigraph enabled
all edge kinds equal to oxygen_bridge
```

No Na or O node appears in projected mode.

Atomic-path acceptance view

The diagnostic view must contain:

```
48 T-site vertex nodes
96 used O-linker nodes
144 total source nodes
192 T-O path-segment edges
0 Na nodes
```

Every parent projected edge contributes two diagnostic segments.

Required artifacts

Acceptance generation should produce:

1. projected 2-D PCA PNG;
2. projected 2-D local-neighborhood PNG;
3. projected canonical-cell 3-D HTML;
4. projected expanded $2 \times 2 \times 1$ 3-D HTML;
5. atomic-path canonical-cell 3-D HTML;
6. an integration report containing source and display counts.

Three-dimensional artifacts are delivered as files or links, not inline previews.

Implementation plan

Phase FVG-1: API and validation

- implement `FrameworkGraphDisplayMode`;
- implement `FrameworkPathSegmentKey`;
- validate collection/topology compatibility;
- export the public API.

Phase FVG-2: periodic geometry

- reconstruct retained segment minimum-image shifts;
- reconstruct projected frame shifts through a deterministic vertex gauge;
- verify canonical winding equivalence;
- record display warnings.

Phase FVG-3: graph adapters

- build projected graph views;
- build atomic-path diagnostic views;
- populate stable metadata columns;
- preserve multigraph semantics.

Phase FVG-4: style and wrappers

- implement `GraphStyle.framework_default`;
- implement 2-D and 3-D convenience wrappers;
- retain generic user overrides.

Phase FVG-5: tests and integration

- synthetic single-linker path;
- direct edge;

- periodic path crossing;
- parallel projected paths;
- incompatible frame rejection;
- serialization of diagnostic segment keys;
- Na-LTA projected and atomic-path counts;
- 2-D raster export and 3-D HTML export.

Phase FVG-6: source/specification audit

- compare public signatures with this document;
- compare dataclass fields;
- verify exports;
- regenerate the PDF;
- preflight and render-inspect the PDF;
- run the complete package regression suite.

Future extension points

Later adapters may add:

- ring nodes and ring-center geometry;
- projected edges rendered as curved or multi-segment atomic paths;
- edge selection linked between projected and diagnostic views;
- cage and site overlays;
- trajectory animation.

These features must reuse `DecoratedGraphView` and must not change the scientific identity of `FrameworkTopology`.